
PILLAR 6: Spring Framework & Spring Boot Ecosystem

6.1 IoC and Dependency Injection

Clear Definition & Internal Mechanics

Inversion of Control (IoC) is a design principle where the *framework* (Spring) controls the lifecycle and wiring of objects, rather than application code using `new` directly. The specific implementation pattern Spring uses is **Dependency Injection (DI)**.

DI Mechanics: Instead of a class creating its dependencies:

```
java

class OrderService {
    PaymentService paymentService = new CreditCardPaymentService(); // tight coupling
}
```

Spring creates both beans and *injects* the dependency:

```
java

@Service
class OrderService {
    private final PaymentService paymentService; // interface dependency

    @Autowired // Spring injects the concrete implementation
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Three DI Types:

1. **Constructor Injection** (recommended): Dependencies provided at construction time. Enables immutability (`final` fields) and makes dependencies explicit. Facilitates testing without Spring.
2. **Setter Injection**: Optional dependencies. Less common; allows circular dependency workarounds (though circular dependencies indicate design issues).

3. **Field Injection** (`@Autowired` on a field): Convenient but hides dependencies, prevents `final`, and requires reflection to inject — making unit testing harder. **Avoid in production code.**

Spring's BeanFactory & IoC Container: The central interface is `ApplicationContext`, which extends `BeanFactory`. At startup, Spring:

1. Scans the classpath for `@Component`-annotated classes (or processes `@Bean` methods in `@Configuration` classes).
2. Builds a **Bean Definition Registry**: a map of bean names → bean definitions (class, scope, dependencies).
3. **Resolves the dependency graph**: Performs topological sort of dependencies.
4. **Instantiates beans** in dependency order using reflection (`Constructor.newInstance()`).
5. **Injects dependencies**, runs `@PostConstruct`, registers with `ApplicationContext`.

Real-Life Analogy

Without IoC, you're a **chef who shops for ingredients, cooks, and plates the dish entirely yourself** — you control every step. With IoC, you're a chef at a high-end restaurant: a **kitchen manager** (Spring) sources all ingredients (dependencies), stocks your station, and tells you when to start. You focus entirely on cooking (business logic). The control of setup is *inverted* from you to the manager.

Interview Context & "Why it Matters"

IoC is the foundational Spring concept. Every other Spring feature (AOP, transactions, security) is built on it. Interviewers test:

- Why constructor injection is preferred over field injection.
- How circular dependencies manifest and how to break them.
- The difference between `BeanFactory` (lazy) and `ApplicationContext` (eager) initialization.
- Testing implications: constructor-injected beans can be unit tested without Spring context.

Production-Ready Code Snippet

```
java
```

```
import org.springframework.stereotype.Service;
import org.springframework.stereotype.Repository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Primary;

// — Domain Interfaces —————

interface OrderRepository {
    void save(String orderId);
}

interface NotificationService {
    void notify(String message);
}

// — Infrastructure Implementations —————

@Repository
class JpaOrderRepository implements OrderRepository {
    @Override
    public void save(String orderId) {
        System.out.println("[JPA] Persisting order: " + orderId);
    }
}

@Service
@Primary // marks this as the preferred implementation when multiple beans exist
class EmailNotificationService implements NotificationService {
    @Override
    public void notify(String message) {
        System.out.println("[Email] Sending: " + message);
    }
}

@Service
class SmsNotificationService implements NotificationService {
    @Override
    public void notify(String message) {
        System.out.println("[SMS] Sending: " + message);
    }
}

// — Business Logic (Depends on abstractions, not concretions) —————
```

```

@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final NotificationService notificationService;

    // ✅ Constructor Injection: dependencies are explicit, final, testable without
    @Autowired
    public OrderService(OrderRepository orderRepository,
                        @org.springframework.beans.factory.annotation.Qualifier("email")
                        NotificationService notificationService) {
        if (orderRepository == null) throw new IllegalArgumentException("OrderRepository is null");
        if (notificationService == null) throw new IllegalArgumentException("NotificationService is null");
        this.orderRepository = orderRepository;
        this.notificationService = notificationService;
    }

    public void placeOrder(String orderId) {
        if (orderId == null || orderId.isBlank()) {
            throw new IllegalArgumentException("Order ID must not be blank.");
        }
        orderRepository.save(orderId);
        notificationService.notify("Order " + orderId + " placed successfully.");
    }
}

// — Unit Test (No Spring Context Required!) —————
// class OrderServiceTest {
//     @Test
//     void placeOrder_shouldSaveAndNotify() {
//         OrderRepository mockRepo = mock(OrderRepository.class);
//         NotificationService mockNotif = mock(NotificationService.class);
//         OrderService service = new OrderService(mockRepo, mockNotif);
//         service.placeOrder("ORD-001");
//         verify(mockRepo).save("ORD-001");
//         verify(mockNotif).notify("Order ORD-001 placed successfully.");
//     }
// }

```

Common Interview Question & Expert Answer

Q: "What's the difference between `@Autowired` on a field versus a constructor? Which do you

prefer and why?"

A: Both accomplish dependency injection, but constructor injection is strictly superior for production code:

Aspect	Field Injection	Constructor Injection
Immutability	✗ Cannot use <code>final</code>	✓ Fields can be <code>final</code>
Testability	✗ Requires Spring context or reflection	✓ Plain <code>new</code> in tests
Null safety	✗ Fails at runtime	✓ Fails at construction (early)
Visibility	✗ Dependencies hidden	✓ Dependencies explicit in signature
Circular deps	Partially hides them	Exposes them as compilation error

Field injection uses reflection to bypass access modifiers and set private fields — it works but is an anti-pattern because it creates hidden coupling and makes the class's contract opaque. With constructor injection, the fact that `OrderService` needs an `OrderRepository` is **expressed in its type signature**, making dependencies self-documenting. In tests, you pass mocks directly via the constructor without any Spring magic.

Spring's own documentation since 4.x recommends constructor injection as the default, with setter injection only for optional dependencies.

6.2 ApplicationContext & Beans

Clear Definition & Internal Mechanics

The **ApplicationContext** is Spring's central IoC container interface. It extends `BeanFactory` and adds:

- **Event publishing** (`ApplicationEventPublisher`)
- **Internationalization** (`MessageSource`)
- **Resource loading** (`ResourceLoader`)
- **AOP integration**
- **Eager initialization** of singleton beans (vs. `BeanFactory`'s lazy initialization)

Bean Lifecycle (Detailed):

Container Starts



1. Bean Definition Loading (classpath scan, @Configuration processing)



2. BeanFactoryPostProcessor runs (e.g., PropertySourcesPlaceholderConfigurer resolves \${properties})



3. Bean Instantiation (via constructor)



4. Dependency Injection (setters, fields, or constructor)



5. BeanPostProcessor.postProcessBeforeInitialization() (e.g., @Autowired validation)



6. @PostConstruct method executes



7. InitializingBean.afterPropertiesSet() (if implemented)



8. @Bean(initMethod="...") custom init method



9. BeanPostProcessor.postProcessAfterInitialization() (e.g., AOP proxy creation)



10. Bean is READY – available via ApplicationContext.getBean()



... Application Runs ...



11. @PreDestroy method executes (on context close)



12. DisposableBean.destroy()



▼
13. @Bean(destroyMethod="...") custom destroy method

BeanPostProcessor: The most powerful extension point. Spring itself uses BPPs for **@Autowired** processing (**AutowiredAnnotationBeanPostProcessor**), **@Transactional** proxy creation, and AOP proxy wrapping.

Real-Life Analogy

The `ApplicationContext` is like a **staffing agency** that manages employees (beans) across their entire employment lifecycle: recruitment (instantiation), onboarding (initialization callbacks), daily work (application running), and offboarding (destruction hooks). The agency also handles interdependencies — ensuring an employee who needs a mentor is onboarded *after* their mentor is ready.

Interview Context & "Why it Matters"

Understanding the bean lifecycle is essential for:

- Knowing when to use **@PostConstruct** vs. constructor initialization.
- Understanding *when* AOP proxies are created (after step 9 — so calling **this.method()** inside a bean bypasses AOP!).
- Debugging startup ordering issues.
- Implementing custom **BeanPostProcessor** for cross-cutting infrastructure concerns.

Production-Ready Code Snippet

```
java
```

```

import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.springframework.beans.factory.InitializingBean;
import org.springframework.beans.factory.DisposableBean;
import org.springframework.stereotype.Component;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

/**
 * Demonstrates the full Spring bean lifecycle with multiple init/destroy hooks.
 * Hooks execute in the documented order.
 */
@Component
public class DatabaseConnectionPool implements InitializingBean, DisposableBean {

    private ExecutorService connectionPool;
    private int poolSize;

    // Step 3: Constructor – Spring has NOT injected dependencies yet.
    // Keep lightweight; do NOT call injected dependencies here.
    public DatabaseConnectionPool() {
        System.out.println("[1] Constructor: Bean instance created.");
        this.poolSize = 10; // default – will be overridden by @Value injection
    }

    // Step 6: @PostConstruct – All dependencies injected.
    // Safe to call injected services, start background tasks.
    @PostConstruct
    public void initializePool() {
        System.out.println("[2] @PostConstruct: Initializing connection pool with "
            + this.poolSize);
        this.connectionPool = Executors.newFixedThreadPool(poolSize,
            r -> {
                Thread t = new Thread(r, "db-pool-worker");
                t.setDaemon(true);
                return t;
            });
    }

    // Step 7: InitializingBean – runs after @PostConstruct.
    // Prefer @PostConstruct; this is for backward compatibility.
    @Override
    public void afterPropertiesSet() {

```



```

        System.out.println("[3] InitializingBean.afterPropertiesSet(): Pool validation complete.");
    }

    public void executeQuery(String sql) {
        if (sql == null || sql.isBlank()) throw new IllegalArgumentException("SQL must not be null or blank");
        connectionPool.submit(() -> System.out.println("[Pool] Executing: " + sql));
    }

    // Step 11: @PreDestroy – context is closing.
    // Release resources, flush buffers, send shutdown signals.
    @PreDestroy
    public void drainPool() {
        System.out.println("[4] @PreDestroy: Draining connection pool.");
        if (connectionPool != null) {
            connectionPool.shutdown();
            try {
                if (!connectionPool.awaitTermination(5, TimeUnit.SECONDS)) {
                    connectionPool.shutdownNow();
                    System.err.println("[4] @PreDestroy: Forced shutdown after timeout.");
                }
            } catch (InterruptedException e) {
                connectionPool.shutdownNow();
                Thread.currentThread().interrupt();
            }
        }
    }

    // Step 12: DisposableBean – runs after @PreDestroy.
    @Override
    public void destroy() {
        System.out.println("[5] DisposableBean.destroy(): Final cleanup complete.");
    }
}

```

Common Interview Question & Expert Answer

Q: "Why does calling `this.myTransactionalMethod()` from within the same bean bypass the `@Transactional` proxy?"

A: Spring's `@Transactional` works by wrapping the bean in a **CGLIB proxy** (or JDK dynamic proxy). When external code calls `orderService.placeOrder()`, it actually calls the *proxy's* `placeOrder()` method. The proxy intercepts the call, opens a transaction, delegates to the real `OrderService.placeOrder()`, then commits/rolls back.

However, when `placeOrder()` internally calls `this.helperMethod()`, `this` refers to the **original, unproxied bean**, not the proxy. The proxy is completely bypassed. So if `helperMethod()` is annotated with `@Transactional(propagation = REQUIRES_NEW)`, that propagation is silently ignored.

Solutions:

1. **Extract `helperMethod()` to a separate Spring bean** and inject it. Calls between beans go through the proxy.
2. **Self-injection** (ugly, but works): Inject the bean into itself (`@Lazy @Autowired` to break circular dependency), then call `self.helperMethod()`.
3. **`AopContext.currentProxy()`** (requires `@EnableAspectJAutoProxy(exposeProxy=true)`): `((OrderService) AopContext.currentProxy()).helperMethod()`.

The cleanest fix is always option 1 — if a method needs different transactional behavior, it belongs in a different service.

6.3 Bean Scopes

Clear Definition & Internal Mechanics

Spring's bean scope defines **how many instances** of a bean exist and **for how long**.

Scope	Description	Instance Count	Availability
Singleton	One instance per Spring container	1	All
Prototype	New instance per <code>getBean()</code> / injection	N	All
Request	One per HTTP request	1 per request	Web only
Session	One per HTTP session	1 per session	Web only
Application	One per <code>ServletContext</code>	1	Web only
WebSocket	One per WebSocket session	1 per WS	Web only

Singleton (Default)

- Created eagerly at context startup.
- Shared across all threads — **must be thread-safe**.
- Stateless services, repositories, and controllers should be Singleton.

- **Concurrent trap:** storing mutable state in a singleton field is a race condition unless synchronized or using thread-safe data structures.

Prototype

- A **new instance** is created every time the bean is requested (`getBean()` or injected).
- Spring creates the bean but **does NOT manage its destruction** — `@PreDestroy` is NOT called. The caller owns the lifecycle.
- Use for stateful, non-thread-safe beans (e.g., parsers, builders, command objects).

Injecting Prototype into Singleton (The Scope Mismatch Problem)

When a Singleton bean has a Prototype dependency injected via constructor/field, the Prototype is only created **once** at the Singleton's creation. Every call to the Singleton uses the *same* Prototype instance — defeating the purpose.

Fixes:

1. **Method Injection** (`@Lookup`): Spring overrides the annotated method to return a fresh Prototype each time.
2. `ApplicationContext.getBean()`: Programmatically fetch a new Prototype (coupled to Spring API).
3. `ObjectProvider<T>` (Spring 4.3+): Lazy, type-safe provider that creates a new instance on each `getObject()` call.

Real-Life Analogy

Singleton is the **company CEO** — one person, shared across all departments. **Prototype** is a **disposable coffee cup** — a fresh one for every use, discarded after. **Request scope** is a **temporary visitor badge** — created when you enter, revoked when you leave the building. The classic trap: giving a CEO a personal assistant who's supposed to be unique per meeting — but because the CEO was hired once, they always use the same assistant regardless of the meeting.

Interview Context & "Why it Matters"

The Singleton + Prototype mismatch is a classic Spring interview trap. Stateful singletons cause race conditions in concurrent environments. Knowing `@Lookup` and `ObjectProvider` distinguishes candidates who've used Spring in production from those who've only read tutorials.

Production-Ready Code Snippet

```
java
```

```

import org.springframework.beans.factory.ObjectProvider;
import org.springframework.beans.factory.annotation.Lookup;
import org.springframework.beans.factory.config.ConfigurableBeanFactory;
import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;
import org.springframework.stereotype.Service;

// — Prototype-scoped request processor (stateful, not thread-safe alone) —

@Component
@Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
public class RequestProcessor {
    private final String id = java.util.UUID.randomUUID().toString();

    public void process(String data) {
        System.out.printf("[RequestProcessor %s] Processing: %s\n", id, data);
    }
}

// — Fix 1: ObjectProvider (preferred Spring 4.3+ approach) —

@Service
public class OrderHandlerWithProvider {
    private final ObjectProvider<RequestProcessor> processorProvider;

    public OrderHandlerWithProvider(ObjectProvider<RequestProcessor> processorProvider) {
        this.processorProvider = processorProvider;
    }

    public void handle(String orderId) {
        if (orderId == null || orderId.isBlank()) throw new IllegalArgumentException();
        // getObject() creates a FRESH Prototype instance each call
        RequestProcessor processor = processorProvider.getObject();
        processor.process(orderId);
    }
}

// — Fix 2: @Lookup method injection —

@Service
public abstract class OrderHandlerWithLookup {

    // Spring overrides this method at runtime (CGLIB) to return new Prototype each

```

```

@Lookup
protected abstract RequestProcessor createProcessor();

public void handle(String orderId) {
    if (orderId == null || orderId.isBlank()) throw new IllegalArgumentException
    RequestProcessor processor = createProcessor(); // always a new instance
    processor.process(orderId);
}
}

// — Singleton with thread-safety consideration —————

@Service
public class CounterService {
    // ❌ WRONG: Mutable state in singleton without synchronization
    // private int requestCount = 0;

    // ✅ CORRECT: Use atomic or concurrent structure for singleton state
    private final java.util.concurrent.atomic.AtomicInteger requestCount =
        new java.util.concurrent.atomic.AtomicInteger(0);

    public int incrementAndGet() {
        return requestCount.incrementAndGet();
    }

    public int getCount() {
        return requestCount.get();
    }
}

```

Common Interview Question & Expert Answer

Q: "If I inject a Prototype bean into a Singleton using `@Autowired`, what happens? Show me two ways to fix it."

A: The Prototype bean is injected **once**, at Singleton creation time. Every subsequent call to the Singleton uses the *same Prototype instance*, making it effectively behave as a Singleton — completely defeating its purpose. Here are two fixes:

Fix 1: `ObjectProvider<T>` — Spring injects a provider (factory) rather than an instance. Call `provider.getObject()` each time you need a fresh instance:

```
java
```

```
@Autowired private ObjectProvider<RequestProcessor> provider;  
// In method: RequestProcessor fresh = provider.getObject(); // new instance each ca
```

Fix 2: `@Lookup` method injection — Declare the class `abstract`, annotate the factory method with `@Lookup`. Spring subclasses the bean via CGLIB and overrides the method to call `getBean(RequestProcessor.class)` each invocation:

```
java  
  
@Lookup protected abstract RequestProcessor createProcessor(); // always new
```

`ObjectProvider` is cleaner and works in non-abstract classes. `@Lookup` is useful when you want the intent expressed as a factory method in the class hierarchy.

6.4 Stereotype Annotations & Classpath Scanning

Clear Definition & Internal Mechanics

Spring's stereotype annotations are **marker annotations** that trigger automatic bean registration during **classpath scanning** (`@ComponentScan`).

Annotation	Extends	Semantic Purpose
<code>@Component</code>	—	Generic Spring-managed component
<code>@Service</code>	<code>@Component</code>	Business logic / service layer
<code>@Repository</code>	<code>@Component</code>	Data access layer; enables PersistenceExceptionTranslation
<code>@Controller</code>	<code>@Component</code>	Spring MVC web layer
<code>@RestController</code>	<code>@Controller</code> + <code>@ResponseBody</code>	REST API endpoints
<code>@Configuration</code>	<code>@Component</code>	Bean definition source (factory class)

Classpath Scanning Mechanics:

- `@EnableAutoConfiguration` / `@SpringBootApplication` triggers `@ComponentScan` rooted at the main class's package.

2. Spring uses `ClassPathScanningCandidateComponentProvider` to scan `.class` files.
3. It reads bytecode (via ASM — not full class loading) to detect `@Component` meta-annotations.
4. Qualifying classes are registered as `BeanDefinition` objects in the `BeanDefinitionRegistry`.
5. After scanning, the bean creation lifecycle proceeds as described in §6.2.

@Repository Special Behavior: Spring automatically wraps `@Repository` beans in a proxy that translates JPA/JDBC exceptions (vendor-specific) into Spring's unified `DataAccessException` hierarchy. This decouples business logic from persistence technology specifics.

@Configuration Proxy: `@Configuration` classes are proxied by CGLIB. When a `@Bean` method calls another `@Bean` method within the same `@Configuration`, the proxy intercepts the call and returns the existing singleton bean rather than creating a new instance. Without `@Configuration` (i.e., using `@Component` with `@Bean`), each call to the method would create a fresh instance (called "**lite**" mode).

Real-Life Analogy

Stereotype annotations are like **name badges at a company**: `@Service` = "Engineering", `@Repository` = "Data & Storage", `@Controller` = "Customer Interface". The receptionist (Spring) scans everyone's badge and routes requests accordingly. `@Configuration` is the "Kitchen Staff" badge — people with this badge follow special rules about preparing dishes (beans) once and sharing them, rather than making a fresh batch each time someone orders.

Interview Context & "Why it Matters"

The `@Configuration` vs `@Component` (lite mode) distinction is a nuanced Spring interview topic. `@Repository`'s exception translation is often overlooked. Understanding the bytecode scanning (ASM-based) vs. class-loading distinction shows deep knowledge.

Production-Ready Code Snippet

```
java
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.stereotype.Repository;
import org.springframework.stereotype.Service;
import java.util.HashMap;
import java.util.Map;
import java.util.Optional;

// — Repository Layer —————

@Repository // Enables exception translation; marks as persistence component
public class ProductRepository {

    // In-memory store simulating a database
    private final Map<Long, String> store = new HashMap<>().of(
        1L, "Laptop",
        2L, "Monitor",
        3L, "Keyboard"
    ));

    public Optional<String> findById(Long id) {
        if (id == null || id <= 0) throw new IllegalArgumentException("ID must be positive");
        return Optional.ofNullable(store.get(id));
    }

    public void save(Long id, String name) {
        if (id == null || name == null || name.isBlank()) {
            throw new IllegalArgumentException("ID and name must be valid.");
        }
        store.put(id, name);
    }
}
```

```
// — Service Layer —————

@Service
public class ProductService {
    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }
}
```



```

        public String getProductName(Long id) {
            return productRepository.findById(id)
                .orElseThrow(() -> new ProductNotFoundException("Product not found: " + id));
        }
    }

class ProductNotFoundException extends RuntimeException {
    public ProductNotFoundException(String message) { super(message); }
}

// — Configuration Class (CGLIB-proxied – @Bean methods return singletons) —

@Configuration
public class AppConfig {

    @Bean
    public ProductRepository productRepository() {
        System.out.println("Creating ProductRepository bean");
        return new ProductRepository();
    }

    @Bean
    public ProductService productService() {
        // This call to productRepository() goes through the CGLIB proxy –
        // it returns the SAME singleton bean, NOT a new instance.
        return new ProductService(productRepository());
    }

    // — Lite Mode Comparison (on a @Component class): —————
    // If AppConfig were @Component instead of @Configuration,
    // productRepository() would create a NEW instance each time – breaking singleton semantics
}

```

Common Interview Question & Expert Answer

Q: "What is the difference between `@Configuration` and `@Component` when used as a source of `@Bean` methods?"

A: In `@Configuration` classes, Spring applies **CGLIB subclassing**. The class is replaced at runtime with a generated subclass that overrides `@Bean` methods. When one `@Bean` method calls another within the same class, the override intercepts the call and returns the existing singleton from the `ApplicationContext` — ensuring singleton semantics.

In `@Component` classes with `@Bean` methods ("lite mode"), there is **no CGLIB proxy**. Calling one `@Bean` method from another is a plain Java method call — it creates a brand new object each time, completely bypassing the singleton registry.

```
java

@Configuration // CGLIB-proxied
class Config {
    @Bean DataSource dataSource() { return new DataSource(); }

    @Bean JdbcTemplate jdbcTemplate() {
        return new JdbcTemplate(dataSource()); // proxy returns the singleton DataSource
    }
}

// vs.

@Component // NO proxy - lite mode
class LiteConfig {
    @Bean DataSource dataSource() { return new DataSource(); }

    @Bean JdbcTemplate jdbcTemplate() {
        return new JdbcTemplate(dataSource()); // NEW DataSource instance - WRONG!
    }
}
```

Always use `@Configuration` for classes that define inter-dependent `@Bean` methods. Use `@Component` with `@Bean` only for standalone, independent factory methods.

6.5 Spring MVC Workflow

Clear Definition & Internal Mechanics

Spring MVC is a **front-controller pattern** implementation where all HTTP requests enter through a single `DispatcherServlet`.

Request Processing Pipeline:

HTTP Request



DispatcherServlet

1. `HandlerMapping.getHandler()` —————
Maps URL + Method to @Controller method
(RequestMappingHandlerMapping is the primary impl)
2. `HandlerAdapter.handle()` —————
Invokes the handler via reflection
(RequestMappingHandlerAdapter)
Includes argument resolution:
 - @PathVariable, @RequestParam → MethodArg
 - @RequestBody → Jackson ObjectMapper.readValue
 - @Valid → Bean Validation (Hibernate Validator)
3. Return Value Handling —————
 - ResponseEntity → sets status + headers
 - Object + @ResponseBody → Jackson serialize
 - ModelAndView → ViewResolver (for MVC templates)
4. HandlerInterceptor chain —————
`preHandle()`, `postHandle()`, `afterCompletion()`
(authentication, logging, metrics)



HTTP Response

HandlerInterceptor vs. Filter:

- **Filter** (Servlet API): Runs before/after **DispatcherServlet**. No access to Spring context. Used for CORS, encoding, security.
- **HandlerInterceptor** (Spring): Runs inside **DispatcherServlet**. Has access to handler method, Spring context. Used for business-level pre/post processing.

Real-Life Analogy

The **DispatcherServlet** is an **airport's central hub**. Every flight (HTTP request) lands at the hub. The hub checks the flight manifest (HandlerMapping) to determine which gate (controller method) to route to. Ground staff (HandlerInterceptors) check tickets before boarding

(preHandle), and collect trash after (postHandle). The jetway (HandlerAdapter) connects the plane to the gate and handles the actual boarding (argument resolution and method invocation). The airline's ticketing system (ViewResolver or MessageConverters) handles the passenger's final paperwork (response serialization).

Interview Context & "Why it Matters"

Understanding the pipeline helps diagnose: why `@Valid` isn't triggering, why a filter isn't seeing Spring Security context, why `@ControllerAdvice` isn't catching certain exceptions. It also underpins API design decisions (content negotiation, media types, status codes).

Production-Ready Code Snippet

```
java
```

```

import org.springframework.http.*;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.HandlerInterceptor;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;

// — Request/Response DTOs —————

record CreateOrderRequest(
    @NotBlank(message = "Product ID is required") String productId,
    @Positive(message = "Quantity must be positive") int quantity
) {}

record OrderResponse(String orderId, String productId, int quantity, String status)

// — Controller —————

@RestController
@RequestMapping("/api/v1/orders")
@Validated
public class OrderController {

    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @PostMapping(
        consumes = MediaType.APPLICATION_JSON_VALUE,
        produces = MediaType.APPLICATION_JSON_VALUE
    )
    public ResponseEntity<OrderResponse> createOrder(
        @Valid @RequestBody CreateOrderRequest request) {

        // orderService.placeOrder() would return a domain result
        OrderResponse response = new OrderResponse(
            java.util.UUID.randomUUID().toString(),
            request.productId(),

```

```

        request.quantity(),
        "CONFIRMED"
    );

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .header("X-Order-Id", response.orderId())
        .body(response);
}

```

```

@GetMapping("/{orderId}")
public ResponseEntity<OrderResponse> getOrder(
    @PathVariable String orderId,
    @RequestParam(defaultValue = "false") boolean verbose) {

    if (orderId == null || orderId.isBlank()) {
        return ResponseEntity.badRequest().build();
    }
    // Simulate lookup
    OrderResponse response = new OrderResponse(orderId, "PROD-001", 2, "SHIPPED")
    return ResponseEntity.ok(response);
}
}

```

// — HandlerInterceptor (Request Timing + Correlation ID) —————

```

public class RequestTracingInterceptor implements HandlerInterceptor {

    private static final String START_TIME_ATTR = "requestStartTime";
    private static final String CORRELATION_HEADER = "X-Correlation-Id";

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response,
        Object handler) {
        request.setAttribute(START_TIME_ATTR, System.currentTimeMillis());
        String correlationId = request.getHeader(CORRELATION_HEADER);
        if (correlationId == null || correlationId.isBlank()) {
            correlationId = java.util.UUID.randomUUID().toString();
        }
        response.setHeader(CORRELATION_HEADER, correlationId);
        return true; // continue processing; return false to abort
    }

    @Override

```

```

public void afterCompletion(HttpServletRequest request, HttpServletResponse response,
                           Object handler, Exception ex) {
    Long startTime = (Long) request.getAttribute(START_TIME_ATTR);
    if (startTime != null) {
        long durationMs = System.currentTimeMillis() - startTime;
        System.out.printf("[Tracing] %s %s → %d (%d ms)%n",
            request.getMethod(), request.getRequestURI(),
            response.getStatus(), durationMs);
    }
}
}
}

```

Common Interview Question & Expert Answer

Q: "When should you use a `Filter` vs. a `HandlerInterceptor`? Give a concrete example for each."

A:

Use a `Filter` when:

- You need to process requests *before* or *after* the `DispatcherServlet` — e.g., for cross-cutting concerns that apply to all web components, not just Spring MVC controllers.
- Examples: CORS headers (must be set before `DispatcherServlet` rejects preflight), request body compression/decompression, IP-based rate limiting, encoding normalization.
- Security filters (Spring Security's `DelegatingFilterProxy`) are Filters because security must run before routing decisions.

Use a `HandlerInterceptor` when:

- You need access to the **specific handler method** that will process the request (e.g., to check method-level annotations for custom authorization).
- You need Spring context awareness (injecting Spring beans into the interceptor).
- Examples: Logging the controller method name + execution time, checking a custom `@RequiresRole` annotation on the handler, populating an audit context with handler metadata.

The key boundary: **Filters are Servlet-container-level** (they see raw `HttpServletRequest`/`HttpServletResponse`). **Interceptors are Spring-level** (they see the resolved handler, have access to `ModelAndView`, and integrate with Spring's request lifecycle).

6.6 RESTful APIs

Clear Definition & Internal Mechanics

REST (Representational State Transfer) is an architectural style built on HTTP semantics. Spring MVC implements REST via `@RestController` (which combines `@Controller` + `@ResponseBody`) and HTTP message converters.

HTTP Method Semantics:

Method	Semantics	Idempotent	Safe	Request Body	Response Body
GET	Read resource	✓	✓	✗	✓
POST	Create resource	✗	✗	✓	✓
PUT	Replace resource	✓	✗	✓	Optional
PATCH	Partial update	✗*	✗	✓	Optional
DELETE	Remove resource	✓	✗	✗	Optional

Message Conversion: Spring uses `HttpMessageConverter` implementations.

`MappingJackson2HttpMessageConverter` handles JSON serialization/deserialization via Jackson `ObjectMapper`. Content negotiation uses the `Accept` header (response) and `Content-Type` header (request body).

HTTP Status Code Best Practices:

Scenario	Status Code
GET success	200 OK
POST creates resource	201 Created (+ Location header)
PUT/PATCH success	200 OK
DELETE success	204 No Content
Validation error	400 Bad Request
Unauthenticated	401 Unauthorized
Forbidden	403 Forbidden
Not found	404 Not Found
Conflict (duplicate)	409 Conflict
Server error	500 Internal Server Error

Real-Life Analogy

A REST API is like a **well-organized library system** with standard request forms. GET is "May I read book X?" (safe, can repeat). POST is "Please add this new book to the collection" (creates). PUT is "Replace book X with this edition" (full replacement, idempotent). PATCH is "Correct the typo on page 42 of book X" (partial). DELETE is "Remove book X from circulation." The librarian (server) uses standard HTTP status codes (HTTP 200 = "Here's your book", HTTP 404 = "We don't have that book").

Interview Context & "Why it Matters"

REST API design is fundamental to backend engineering. Interview questions focus on: idempotency design, versioning strategies (URI vs. header), proper status codes, pagination design, HATEOAS, and error response format standardization.

Production-Ready Code Snippet

```
java
```

```

import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;
import jakarta.validation.constraints.*;
import java.net.URI;
import java.util.Map;

// — API-level DTOs with validation —————

record CreateProductRequest(
    @NotBlank(message = "Name is required") @Size(max = 255) String name,
    @NotNull(message = "Price is required") @Positive Double price,
    @NotBlank String category
) {}

record UpdateProductRequest(
    @Size(max = 255) String name,
    @Positive Double price
) {}

record ProductDto(Long id, String name, double price, String category) {}

record ApiError(int status, String message, java.time.Instant timestamp) {
    static ApiError of(int status, String message) {
        return new ApiError(status, message, java.time.Instant.now());
    }
}

// — REST Controller —————

@RestController
@RequestMapping("/api/v1/products")
public class ProductController {

    // GET /api/v1/products?page=0&size=20&sort=name,asc
    @GetMapping(produces = MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity<Map<String, Object>> getProducts(
        @PageableDefault(size = 20, sort = "name") Pageable pageable,
        @RequestParam(required = false) String category) {

```

```

// In real code: productService.findAll(category, pageable)
// Returning minimal pagination envelope for demo
Map<String, Object> response = Map.of(
    "content", java.util.List.of(new ProductDto(1L, "Laptop", 999.99, "Electronics")),
    "page", pageable.getPageNumber(),
    "size", pageable.getPageSize(),
    "totalElements", 1
);
return ResponseEntity.ok(response);
}

// GET /api/v1/products/{id}
@GetMapping(value =("/{id}", produces = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<ProductDto> getProduct(@PathVariable @Positive Long id) {
    // Simulate: productService.findById(id).orElseThrow(...)
    ProductDto product = new ProductDto(id, "Laptop", 999.99, "Electronics");
    return ResponseEntity.ok(product);
}

// POST /api/v1/products
@PostMapping(consumes = MediaType.APPLICATION_JSON_VALUE,
             produces = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<ProductDto> createProduct(
    @Valid @RequestBody CreateProductRequest request) {

    ProductDto created = new ProductDto(100L, request.name(), request.price(), request.category());

    URI location = org.springframework.web.servlet.support.ServletUriComponentsBuilder
        .fromCurrentRequest()
        .path("/{id}")
        .buildAndExpand(created.id())
        .toUri();

    return ResponseEntity.created(location).body(created);
}

// PATCH /api/v1/products/{id}
@PatchMapping(value =("/{id}", consumes = MediaType.APPLICATION_JSON_VALUE,
                    produces = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<ProductDto> updateProduct(
    @PathVariable @Positive Long id,
    @Valid @RequestBody UpdateProductRequest request) {

    // Only update non-null fields

```

```

        ProductDto updated = new ProductDto(
            id,
            request.name() != null ? request.name() : "Existing Name",
            request.price() != null ? request.price() : 999.99,
            "Electronics"
        );
        return ResponseEntity.ok(updated);
    }

    // DELETE /api/v1/products/{id}
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable @Positive Long id) {
        // productService.delete(id);
        return ResponseEntity.noContent().build(); // 204 No Content
    }
}

```

Common Interview Question & Expert Answer

Q: "What is idempotency in REST, and why does it matter? Is PATCH idempotent?"

A: An operation is **idempotent** if performing it multiple times produces the same result as performing it once. This matters for resilience: clients can safely retry idempotent requests after a network timeout without worrying about duplicate effects.

- **GET, PUT, DELETE:** Idempotent. `DELETE /users/123` multiple times — after the first, the user is gone; subsequent calls return 404 but have no additional side effects.
- **POST:** Not idempotent. `POST /orders` twice creates two orders.
- **PATCH:** *Not necessarily* idempotent — it depends on the implementation. If PATCH sets `{ "quantity": 5 }`, repeating it is idempotent. But if PATCH increments `{ "quantityDelta": +1 }`, repeating it is not. The HTTP spec says PATCH is neither safe nor idempotent by default.

Why it matters in production: Load balancers may retry on timeout. Clients may retry on connection failure. If PUT replaces a resource with the same data, no harm done. If POST creates an order on retry, you get a duplicate charge. Design around this by: using POST for order creation with an **idempotency key** header, or using PUT with a client-generated UUID. Stripe's API does exactly this — every POST request accepts an `Idempotency-Key` header, and the server deduplicates based on it.

6.7 Global Controller Advice

Clear Definition & Internal Mechanics

`@ControllerAdvice` is a specialization of `@Component` that allows a class to be shared across multiple `@Controller` classes. It acts as a **global interceptor** for:

1. `@ExceptionHandler`: Catches exceptions thrown by any `@Controller` and returns a structured HTTP response. Without this, uncaught exceptions result in Spring's default error page or a raw 500 response.
2. `@InitBinder`: Customizes data binding for all controllers (e.g., registering custom property editors, date format converters).
3. `@ModelAttribute`: Adds model attributes available to all controllers (useful in MVC template rendering).

Exception Resolution Chain: When an exception escapes a controller method, `DispatcherServlet` calls `HandlerExceptionResolver` implementations in order:

1. `ExceptionHandlerExceptionResolver`: Looks for `@ExceptionHandler` in the controller, then in `@ControllerAdvice` beans.
2. `ResponseStatusExceptionHandler`: Handles `@ResponseStatus`-annotated exceptions.
3. `DefaultHandlerExceptionResolver`: Handles standard Spring MVC exceptions (404, 405, 415, etc.).

Exception Handler Specificity: If both a controller-local `@ExceptionHandler` and a global `@ControllerAdvice @ExceptionHandler` handle the same exception type, the **controller-local handler takes priority**.

Real-Life Analogy

`@ControllerAdvice` is the **company's HR emergency protocol**. If an incident occurs in any department (exception thrown in any controller), instead of each department manager improvising a response, the HR protocol (global handler) kicks in with a standardized response format. However, if a department has its own well-defined escalation policy (local `@ExceptionHandler`), they handle it internally first.

Interview Context & "Why it Matters"

Proper error handling is critical for API consumers. Unhandled exceptions that leak stack traces expose security vulnerabilities. Inconsistent error response formats break API contracts.

`@ControllerAdvice` is the production standard for centralized, consistent error handling.

Interviewers check whether candidates know how to return `RFC 7807 Problem Details` format.

Production-Ready Code Snippet

java

```

import org.springframework.http.*;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.context.request.WebRequest;
import java.time.Instant;
import java.util.Map;
import java.util.stream.Collectors;

// — Standard error response shape (RFC 7807 inspired) —————

record ProblemDetail(
    String type,        // URI identifying the problem type
    String title,       // Short, human-readable summary
    int status,         // HTTP status code
    String detail,      // Human-readable explanation of this occurrence
    String instance,    // URI of the specific request
    Instant timestamp,
    Map<String, String> errors // field-level validation errors (optional)
) {
    static ProblemDetail of(HttpStatus status, String detail, String instance) {
        return new ProblemDetail(
            "https://api.example.com/errors/" + status.value(),
            status.getReasonPhrase(),
            status.value(),
            detail,
            instance,
            Instant.now(),
            null
        );
    }
}

// — Domain Exceptions —————

class ResourceNotFoundException extends RuntimeException {
    private final String resourceType;
    private final Object resourceId;

    public ResourceNotFoundException(String resourceType, Object resourceId) {
        super(resourceType + " not found with id: " + resourceId);
        this.resourceType = resourceType;
        this.resourceId = resourceId;
    }
}

```

```

    }
    public String getResourceType() { return resourceType; }
    public Object getResourceId() { return resourceId; }
}

class BusinessRuleViolationException extends RuntimeException {
    public BusinessRuleViolationException(String message) { super(message); }
}

// — Global Exception Handler —————

@RestControllerAdvice // = @ControllerAdvice + @ResponseBody
public class GlobalExceptionHandler {

    // Handles @Valid / @Validated validation failures
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ProblemDetail> handleValidationFailure(
        MethodArgumentNotValidException ex, WebRequest request) {

        Map<String, String> fieldErrors = ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .collect(Collectors.toMap(
                FieldError::getField,
                fe -> fe.getDefaultMessage() != null ? fe.getDefaultMessage() : "Invalid"
                (existing, replacement) -> existing // keep first error per field
            ));

        ProblemDetail problem = new ProblemDetail(
            "https://api.example.com/errors/validation",
            "Validation Failed",
            HttpStatus.BAD_REQUEST.value(),
            "One or more fields failed validation.",
            request.getDescription(false),
            Instant.now(),
            fieldErrors
        );
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(problem);
    }

    // Domain-level "not found" → 404
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ProblemDetail> handleNotFound(
        ResourceNotFoundException ex, WebRequest request) {

```



```

        ProblemDetail problem = ProblemDetail.of(
            HttpStatus.NOT_FOUND,
            ex.getMessage(),
            request.getDescription(false)
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(problem);
    }

    // Business rule violations → 409 Conflict
    @ExceptionHandler(BusinessRuleViolationException.class)
    public ResponseEntity<ProblemDetail> handleBusinessRuleViolation(
        BusinessRuleViolationException ex, WebRequest request) {

        ProblemDetail problem = ProblemDetail.of(
            HttpStatus.CONFLICT,
            ex.getMessage(),
            request.getDescription(false)
        );
        return ResponseEntity.status(HttpStatus.CONFLICT).body(problem);
    }

    // Catch-all – never expose internal stack traces
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ProblemDetail> handleUnexpected(
        Exception ex, WebRequest request) {

        // Log the real exception internally (don't send to client)
        System.err.println("[ERROR] Unhandled exception: " + ex.getMessage());
        ex.printStackTrace();

        ProblemDetail problem = ProblemDetail.of(
            HttpStatus.INTERNAL_SERVER_ERROR,
            "An unexpected error occurred. Please try again later.",
            request.getDescription(false)
        );
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(problem);
    }
}

```

Common Interview Question & Expert Answer

Q: "How does `@ControllerAdvice` know which exceptions to intercept? Can you narrow its

scope to specific packages?"

A: By default, `@ControllerAdvice` applies globally to all `@Controller` beans in the application context. Scope can be narrowed in three ways:

1. **By annotation:** `@ControllerAdvice(annotations = RestController.class)` — applies only to controllers annotated with `@RestController`.
2. **By base package:** `@ControllerAdvice("com.example.api.orders")` — applies only to controllers in that package (and subpackages).
3. **By assignable type:** `@ControllerAdvice(assignableTypes = {OrderController.class, PaymentController.class})` — applies only to specific controller classes.

When multiple `@ControllerAdvice` beans handle the same exception type, Spring's ordering rules apply: `@Order` annotation or `Ordered` interface determines precedence. Lower order value = higher priority. A common pattern is to have a domain-specific `@ControllerAdvice` for fine-grained handling and a low-priority global catch-all `@ControllerAdvice` as a safety net.

6.8 Aspect-Oriented Programming (AOP)

Clear Definition & Internal Mechanics

AOP addresses **cross-cutting concerns** — logic that spans multiple modules but doesn't belong in any single one (logging, security, transactions, metrics, caching). Rather than scattering this logic across classes, AOP modularizes it into **Aspects**.

Core Concepts:

Term	Definition
Aspect	A class encapsulating a cross-cutting concern (<code>@Aspect</code>)
Join Point	A point in program execution (method call, field access). Spring AOP supports <i>method execution</i> only
Pointcut	An expression selecting which Join Points to intercept
Advice	Code that runs at a Join Point (<code>@Before</code> , <code>@After</code> , <code>@Around</code> , <code>@AfterReturning</code> , <code>@AfterThrowing</code>)
Target Object	The original bean being proxied
Proxy	The Spring-generated wrapper around the target (CGLIB or JDK dynamic proxy)
Weaving	Linking aspects to the application code — Spring does this at <i>runtime</i> via proxies

Pointcut Expression Language:

```

execution(modifiers? return-type declaring-type? method-name(params) throws?)

execution(* com.example.service.*.*(..))
    ^-- any return type, any class in service package, any method, any params

@annotation(com.example.annotation.Audited)
    ^-- any method annotated with @Audited

```

Advice Types:

- `@Before`: Runs before the method. Cannot prevent execution (use `@Around` for that).
- `@AfterReturning`: Runs after successful return. Can access return value.
- `@AfterThrowing`: Runs when method throws. Can access the exception.
- `@After` (finally): Runs regardless of outcome.
- `@Around`: Most powerful — completely wraps execution. Must call `ProceedingJoinPoint.proceed()` to continue.

Proxy Mechanism:

- If the target implements an interface → JDK Dynamic Proxy (proxies the interface).
- If the target is a concrete class → CGLIB subclass proxy.

- `@EnableAspectJAutoProxy(proxyTargetClass=true)` forces CGLIB for all.
- **Self-invocation limitation** (same as `@Transactional`): calling `this.method()` bypasses the proxy.

Real-Life Analogy

AOP is like a **building's security camera + alarm system**. You don't want every employee to manually log "entering room", "leaving room" — that would clutter their work. Instead, the building infrastructure (Aspect) automatically logs all entry/exits (Join Points) matching certain corridors (Pointcut), without any employee needing to do anything. The employees (business logic) remain unaware of the observation.

Interview Context & "Why it Matters"

AOP is the mechanism behind Spring's most critical features: `@Transactional`, `@Cacheable`, `@Async`, `@Secured`. Understanding how proxies work explains self-invocation limitations, proxy class naming in stack traces, and why `@Transactional` on private methods doesn't work.

Production-Ready Code Snippet

```
java
```

```

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.*;
import org.springframework.stereotype.Component;
import java.util.Arrays;

// — Custom Annotation for Audit Logging —————

import java.lang.annotation.*;
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@interface Audited {
    String action() default "";
}

// — Performance + Audit Aspect —————

@Aspect
@Component
public class OperationalAspect {

    // — @Around: Performance timing for all service methods —————

    @Around("execution(* com.example.service..*(..))")
    public Object measureExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
        long startNs = System.nanoTime();
        String methodName = joinPoint.getSignature().toShortString();

        try {
            Object result = joinPoint.proceed(); // delegate to actual method
            long durationMs = (System.nanoTime() - startNs) / 1_000_000;
            System.out.printf("[PERF] %s completed in %d ms%n", methodName, durationMs);
            return result;
        } catch (Throwable ex) {
            long durationMs = (System.nanoTime() - startNs) / 1_000_000;
            System.err.printf("[PERF] %s FAILED after %d ms: %s%n",
                methodName, durationMs, ex.getMessage());
            throw ex; // re-throw to preserve original exception flow
        }
    }

    // — @Before: Log entry for methods annotated with @Audited —————

    @Before("@annotation(audited)")

```

```

public void auditEntry(org.aspectj.lang.JoinPoint joinPoint, Audited audited) {
    String action = audited.action().isBlank()
        ? joinPoint.getSignature().getName()
        : audited.action();
    System.out.printf("[AUDIT] Action='%s' | Method=%s | Args=%s%n",
        action,
        joinPoint.getSignature().toShortString(),
        Arrays.toString(joinPoint.getArgs()));
}

// — @AfterReturning: Log return values for repository calls —————

@AfterReturning(
    pointcut = "execution(* com.example.repository.*.*(..))",
    returning = "result"
)
public void logRepositoryResult(org.aspectj.lang.JoinPoint jp, Object result) {
    System.out.printf("[REPO] %s returned: %s%n",
        jp.getSignature().toShortString(), result);
}

// — @AfterThrowing: Alert on service-layer exceptions —————

@AfterThrowing(
    pointcut = "execution(* com.example.service.*.*(..))",
    throwing = "ex"
)
public void alertOnException(org.aspectj.lang.JoinPoint jp, Exception ex) {
    System.err.printf("[ALERT] Exception in %s: %s: %s%n",
        jp.getSignature().toShortString(),
        ex.getClass().getSimpleName(),
        ex.getMessage());
    // In production: publish to PagerDuty/OpsGenie metric or send alert
}

// — Service using @Audited annotation —————

@Service // In a real Spring app
class UserService {

    @Audited(action = "CREATE_USER")
    public String createUser(String username, String email) {
        if (username == null || username.isBlank()) throw new IllegalArgumentException

```

```
// Simulate creation
return "user-" + java.util.UUID.randomUUID();
}
}
```

Common Interview Question & Expert Answer

Q: "Why doesn't `@Transactional` work on private methods, and why doesn't it work when a method calls another method in the same class?"

A: Both issues stem from Spring's proxy-based AOP implementation:

1. **Private methods:** Spring creates a CGLIB subclass of your bean class. CGLIB subclasses cannot override `private` methods — they're not visible to the subclass. Therefore, Spring cannot intercept private method calls. The `@Transactional` advice is never applied. (AspectJ compile-time or load-time weaving *can* handle private methods, but that's rarely used in Spring Boot apps.)
2. **Self-invocation:** When `placeOrder()` calls `this.chargeCard()` and `chargeCard()` is `@Transactional(propagation=REQUIRES_NEW)`, `this` refers to the *original bean*, not the Spring proxy. The proxy is the transaction-aware wrapper, but it only intercepts calls from *outside* the proxy. Internal calls skip the proxy entirely.

Both solutions are the same:

- Extract `chargeCard()` to a separate `@Service` bean, inject it, and call it via the injected reference (goes through the proxy).
- Use `@Lazy @Autowired` self-injection (antipattern but functional).
- Use `AopContext.currentProxy()` with `@EnableAspectJAutoProxy(exposeProxy=true)` (brittle).

The root fix is **refactoring**: if `chargeCard()` needs different transactional behavior, it belongs in a `PaymentService`, not the same class as `placeOrder()`. This also improves cohesion.

6.9 Spring Data JPA / Hibernate — ORM Fundamentals

Clear Definition & Internal Mechanics

ORM (Object-Relational Mapping) bridges the gap between Java's object model and a relational database. Hibernate is the JPA (Jakarta Persistence API) implementation Spring Data JPA uses.

Entity States:

State	Description	Tracked by Hibernate?
Transient	<code>new Entity()</code> — not associated with any persistence context	✗
Managed	Retrieved via <code>EntityManager</code> or persisted via <code>persist()</code> — in the context	✓
Detached	Was managed, then context was closed or <code>detach()</code> was called	✗
Removed	Scheduled for deletion via <code>remove()</code> — still in context until flush	✓ (deletion pending)

Persistence Context (First-Level Cache): Within a single transaction, Hibernate maintains an in-memory map of all managed entities (identity map). Calling `findById(1L)` twice returns the *same Java object*, not two separate instances. This prevents stale reads within a transaction and avoids redundant DB queries.

Dirty Checking: At transaction commit (flush), Hibernate compares managed entities' current state against their snapshot taken at load time. Any differences trigger an automatic `UPDATE` SQL without explicit `save()` calls. This is called **automatic dirty checking**.

Relationship Annotations:

- `@OneToMany(fetch=FetchType.LAZY)`: The "many" side is loaded on demand (lazy proxy).
- `@ManyToOne(fetch=FetchType.EAGER)`: The "one" side is loaded immediately. Eager is the default for `@ManyToOne`.
- `@ManyToMany`: Join table. Both sides are lazy by default.
- `@OneToOne`: Can be lazy or eager; lazy requires extra bytecode enhancement.

Cascade Types: `CascadeType.PERSIST`, `MERGE`, `REMOVE`, `REFRESH`, `DETACH`, `ALL`. Cascading propagates operations from parent to children automatically.

Real-Life Analogy

Hibernate's Persistence Context is like a **meticulous personal assistant's notebook**. When you ask for a client's file (load an entity), the assistant writes the original details in the notebook (snapshot). As you make notes and changes (mutate the entity), the assistant tracks differences. When you close the meeting (commit the transaction), the assistant automatically sends an update letter to the file room (issues UPDATE SQL) for every changed detail — without you having to explicitly say "please update this."

Interview Context & "Why it Matters"

ORM is a double-edged sword. Used correctly, it eliminates boilerplate persistence code. Used naively, it causes N+1 query disasters, lazy initialization exceptions, and accidental cartesian product queries. Senior engineers are expected to know the gotchas, not just the happy path.

Production-Ready Code Snippet

```
java
```

```

import jakarta.persistence.*;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

// — Entity Classes —————

@Entity
@Table(name = "customers", indexes = {
    @Index(name = "idx_customer_email", columnList = "email", unique = true)
})
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(nullable = false, unique = true, length = 255)
    private String email;

    // LAZY = orders are NOT loaded when Customer is fetched (prevents N+1 and large
    @OneToMany(mappedBy = "customer", cascade = CascadeType.ALL,
        fetch = FetchType.LAZY, orphanRemoval = true)
    private List<Order> orders = new java.util.ArrayList<>();

    protected Customer() {} // JPA requires no-arg constructor

    public Customer(String name, String email) {
        if (name == null || name.isBlank()) throw new IllegalArgumentException("Name
        if (email == null || !email.contains("@")) throw new IllegalArgumentException
        this.name = name;
        this.email = email;
    }

    // Domain method – keeps invariants inside the entity
    public void addOrder(Order order) {
        if (order == null) throw new IllegalArgumentException("Order must not be nul

```

```
        orders.add(order);  
        order.setCustomer(this);  
    }  
  
    public Long getId() { return id; }  
    public String getName() { return name; }  
    public String getEmail() { return email; }  
    public List<Order> getOrders() { return java.util.Collections.unmodifiableList(o  
    }  
}
```

@Entity

@Table(name = "orders")

public class Order {

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

private Long id;

@ManyToOne(fetch = FetchType.LAZY) // LAZY to avoid N+1 when loading orders

@JoinColumn(name = "customer_id", nullable = false)

private Customer customer;

@Column(nullable = false)

private double totalAmount;

@Enumerated(EnumType.STRING)

@Column(nullable = false)

private OrderStatus status;

@Column(nullable = false)

private LocalDateTime createdAt;

protected Order() {}

public Order(double totalAmount) {

if (totalAmount < 0) throw new IllegalArgumentException("Total amount cannot

this.totalAmount = totalAmount;

this.status = OrderStatus.PENDING;

this.createdAt = LocalDateTime.now();

}

void setCustomer(Customer customer) { this.customer = customer; }

public Long getId() { return id; }

public double getTotalAmount() { return totalAmount; }

```

    public OrderStatus getStatus() { return status; }
    public void confirm() { this.status = OrderStatus.CONFIRMED; }
}

enum OrderStatus { PENDING, CONFIRMED, SHIPPED, CANCELLED }

// — Spring Data JPA Repository —————

@Repository
public interface CustomerRepository extends JpaRepository<Customer, Long> {

    Optional<Customer> findByEmail(String email);

    // JOIN FETCH avoids N+1 – loads customer + orders in one query
    @Query("SELECT c FROM Customer c LEFT JOIN FETCH c.orders WHERE c.id = :id")
    Optional<Customer> findByIdWithOrders(Long id);

    // Specification/Criteria API would be used for dynamic filters
    List<Customer> findByNameContainingIgnoreCase(String namePart);
}

```

Common Interview Question & Expert Answer

Q: "What is the difference between `fetch = FetchType.LAZY` and `EAGER`? Which should you use by default and why?"

A:

- **EAGER:** Related entities are fetched *immediately* when the parent is loaded, in the same query (or an additional query/join). This can cause performance disasters when loading a list of 100 customers where each has 50 orders — you've now loaded 5,000 order records unexpectedly.
- **LAZY:** A proxy is returned for the related collection. The actual data is only fetched when the collection is *first accessed*. This requires an open EntityManager/Session at access time.

Default rule: Always use LAZY for collections (`@OneToMany`, `@ManyToMany`). `@ManyToOne` defaults to EAGER (JPA spec), but switching it to LAZY is also recommended for large graphs.

The LazyInitializationException trap: Accessing a lazy collection *outside* a transaction (after the EntityManager is closed) throws `LazyInitializationException`. Fix options:

1. Ensure access within the transaction (in the service layer).
2. Use `JOIN FETCH` in the JPQL query to eagerly load only when needed.
3. Use `@EntityGraph` to declaratively specify what to fetch per query.

4. Use Spring's `@Transactional(readOnly=true)` on read-only service methods to keep the session open through controller serialization (OpenSessionInView — but this is an anti-pattern for APIs).

The correct approach is **JOIN FETCH** or **@EntityGraph** at the repository query level, tailored to each use case, rather than global EAGER loading.

6.10 Transaction Management

Clear Definition & Internal Mechanics

Transactions ensure database operations execute with ACID properties:

- **Atomicity:** All or nothing.
- **Consistency:** Data moves from one valid state to another.
- **Isolation:** Concurrent transactions don't see each other's partial state.
- **Durability:** Committed changes survive crashes.

Spring's `@Transactional` is **declarative transaction management** via AOP proxy. When a `@Transactional` method is called through a proxy, Spring:

1. Calls `PlatformTransactionManager.getTransaction()` to start or join a transaction.
2. Binds the connection to the current thread via `TransactionSynchronizationManager` (a `ThreadLocal`-backed registry).
3. On successful return: calls `commit()`.
4. On `RuntimeException` or `Error`: calls `rollback()` (by default — checked exceptions do NOT trigger rollback by default).

Propagation Behaviors:

Isolation Levels:

Level	Dirty Read	Non-Repeatable Read	Phantom Read
READ_UNCOMMITTED	✓ possible	✓ possible	✓ possible
READ_COMMITTED	✗ prevented	✓ possible	✓ possible
REPEATABLE_READ	✗ prevented	✗ prevented	✓ possible
SERIALIZABLE	✗ prevented	✗ prevented	✗ prevented

Most production systems use `READ_COMMITTED` (PostgreSQL, MySQL default for InnoDB in `REPEATABLE_READ`).

Real-Life Analogy

A transaction is like a **bank teller's drawer reconciliation**. The teller starts a shift (begins transaction). All cash movements (DB operations) are tracked. At end of shift: if everything balances (no exception), the teller signs off (commit). If there's a discrepancy (exception), the manager resets the drawer to the start-of-shift state (rollback). The drawer's state is never left half-updated.

Production-Ready Code Snippet

```
java
```

```

import org.springframework.transaction.annotation.Isolation;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.stereotype.Service;

@Service
public class TransferService {

    private final AccountRepository accountRepository;
    private final TransactionAuditRepository auditRepository;

    public TransferService(AccountRepository accountRepository,
                           TransactionAuditRepository auditRepository) {
        this.accountRepository = accountRepository;
        this.auditRepository = auditRepository;
    }

    // — Main business operation —————
    // REQUIRED: joins existing or creates new. Rolls back on RuntimeException.
    @Transactional(
        propagation = Propagation.REQUIRED,
        isolation = Isolation.READ_COMMITTED,
        rollbackFor = Exception.class, // also rollback for checked exceptions
        timeout = 30 // seconds; throws TransactionTimedOutException if exceeded
    )
    public void transfer(Long fromAccountId, Long toAccountId, double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Transfer amount must be > 0");
        if (fromAccountId.equals(toAccountId)) throw new IllegalArgumentException("Cannot transfer to self");

        Account from = accountRepository.findByIdWithLock(fromAccountId)
            .orElseThrow(() -> new ResourceNotFoundException("Account", fromAccountId));
        Account to = accountRepository.findByIdWithLock(toAccountId)
            .orElseThrow(() -> new ResourceNotFoundException("Account", toAccountId));

        if (from.getBalance() < amount) {
            throw new BusinessRuleViolationException("Insufficient funds: balance=" + from.getBalance());
        }

        from.debit(amount);
        to.credit(amount);
        // Dirty checking: Hibernate automatically issues UPDATE SQL at commit
        // No explicit save() needed for managed entities
    }
}

```

```

        // Audit log written in SAME transaction – rolls back together if transfer f
        recordAudit(fromAccountId, toAccountId, amount);
    }

    // — REQUIRES_NEW: audit always written, even if outer transaction rolls back

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void recordFailedTransferAttempt(Long fromId, Long toId, double amount, S
        // This runs in its own transaction – survives outer rollback
        auditRepository.saveFailure(fromId, toId, amount, reason);
    }

    // — Read-only transaction: hint to DB to use replica; no dirty checking —

    @Transactional(readOnly = true)
    public double getBalance(Long accountId) {
        return accountRepository.findById(accountId)
            .orElseThrow(() -> new ResourceNotFoundException("Account", accountId))
            .getBalance();
    }

    // Private – called within the same proxy method; inherits transaction (REQUIRED
    private void recordAudit(Long fromId, Long toId, double amount) {
        auditRepository.save(new TransferAudit(fromId, toId, amount));
    }
}

// Stub interfaces to illustrate the pattern
interface AccountRepository {
    java.util.Optional<Account> findById(Long id);
    java.util.Optional<Account> findByIdWithLock(Long id); // SELECT ... FOR UPDATE
}

interface TransactionAuditRepository {
    void save(TransferAudit audit);
    void saveFailure(Long from, Long to, double amount, String reason);
}

class Account {
    private Long id; private double balance;
    Account(Long id, double balance) { this.id = id; this.balance = balance; }
    double getBalance() { return balance; }
    void debit(double amt) { balance -= amt; }
    void credit(double amt) { balance += amt; }
}

class TransferAudit {

```



```
TransferAudit(Long from, Long to, double amount) {}  
}
```

Common Interview Question & Expert Answer

Q: "You have a service method annotated with `@Transactional`. A `RuntimeException` is thrown halfway through. You see some database changes were committed before the exception. What went wrong?"

A: Several possible causes:

1. **Explicit flush before exception:** If `entityManager.flush()` is called (or Hibernate auto-flushes before a query), changes are written to the DB but the transaction isn't committed yet. These are rolled back on exception. However, if the code called `entityManager.flush()` and *then* committed manually, that portion is committed permanently.
2. **REQUIRES_NEW nested transaction committed before exception:** If the call stack passed through a method with `@Transactional(propagation = REQUIRES_NEW)`, that inner transaction was committed independently. When the outer transaction rolls back, the REQUIRES_NEW work is NOT rolled back.
3. **Non-transactional repository call:** A repository method bypassed Spring's transaction by being called without going through the proxy (e.g., through `this.` in the same class).
4. **@Transactional not applied:** Common causes — the method is private, it's called from within the same class (self-invocation), or the class isn't a Spring-managed bean.
5. **Wrong rollback rules:** By default, Spring only rolls back for `RuntimeException` and `Error`. If your code throws a checked exception and `rollbackFor` isn't set, the transaction commits despite the exception.

To diagnose: enable SQL logging (`spring.jpa.show-sql=true`), `logging.level.org.hibernate.SQL=DEBUG`) and transaction logging (`logging.level.org.springframework.transaction=DEBUG`) to see exactly what SQL executed in which transaction boundary.

6.11 The N+1 Select Problem

Clear Definition & Internal Mechanics

The N+1 problem occurs when loading N parent entities triggers N additional queries to load each parent's lazy children — instead of a single joined query.

Classic Example: Loading 100 customers and then accessing each customer's orders:

```
Query 1: SELECT * FROM customers                                → 100 rows
Query 2: SELECT * FROM orders WHERE customer_id = 1
Query 3: SELECT * FROM orders WHERE customer_id = 2
...
Query 101: SELECT * FROM orders WHERE customer_id = 100
```

Total: **101 queries**. At scale (10,000 customers), this becomes catastrophic.

Root Cause: Hibernate's lazy proxy for `@OneToMany` generates one additional SELECT per entity when the collection is accessed in a loop.

Detection:

- Enable Hibernate SQL logging and count queries manually.
- Use **Hibernate Statistics**
(`spring.jpa.properties.hibernate.generate_statistics=true`).
- Use **p6spy** or **datasource-proxy** to count SQL statements per request in tests.
- Use JUnit with `@Sql` + query counting assertions.

Solutions:

Solution	Mechanism	Best For
<code>JOIN FETCH</code>	JPQL <code>LEFT JOIN FETCH</code>	Simple, known associations
<code>@EntityGraph</code>	Declarative fetch plan on repository method	Multiple fetch paths
<code>@BatchSize</code>	Hibernate batch loads N IDs in one IN clause	Large collections
<code>FetchType.EAGER</code>	Always loads — often too broad	Rare, small collections
DTO Projection	Fetch only needed columns via JPQL/DTO	Read-only, complex views

Real-Life Analogy

Imagine a librarian asked to retrieve 100 books and report each book's borrower. Without JOIN FETCH (N+1): the librarian goes to shelf → gets book 1 → goes to card catalog → finds borrower → goes to shelf → gets book 2 → goes to card catalog → finds borrower... 200 trips. With JOIN FETCH: the librarian takes a single sheet listing all books and their borrowers in one trip. Same data, radically fewer steps.

Interview Context & "Why it Matters"

N+1 is arguably the most **common production performance bug** in Spring/Hibernate

applications. It's easy to miss in development (small datasets) and catastrophic in production (large datasets). Interviewers who work on data-intensive systems will almost certainly probe this.

Production-Ready Code Snippet

```
java
```

```

import org.springframework.data.jpa.repository.*;
import org.springframework.stereotype.Repository;
import jakarta.persistence.EntityGraph;
import java.util.List;
import java.util.Optional;

// ——— Fix 1: JOIN FETCH in JPQL —————

@Repository
public interface CustomerRepo extends JpaRepository<Customer, Long> {

    // ❌ N+1 PROBLEM: This loads customers lazily; accessing .getOrders() in a loop
    //                triggers one SELECT per customer
    @Query("SELECT c FROM Customer c")
    List<Customer> findAllLazy();

    // ✅ FIX: JOIN FETCH – single query with LEFT JOIN to orders
    @Query("SELECT DISTINCT c FROM Customer c LEFT JOIN FETCH c.orders")
    List<Customer> findAllWithOrders();

    // ✅ FIX: @EntityGraph – declarative, works with Spring Data method names
    @EntityGraph(attributePaths = {"orders"})
    @Query("SELECT c FROM Customer c WHERE c.email = :email")
    Optional<Customer> findByEmailWithOrders(String email);
}

// ——— Fix 2: @BatchSize on collection (Hibernate-specific) —————

// On the entity mapping:
// @OneToMany(mappedBy = "customer", fetch = FetchType.LAZY)
// @org.hibernate.annotations.BatchSize(size = 50)
// private List<Order> orders;

// With @BatchSize(50), instead of N individual SELECTs,
// Hibernate issues: SELECT * FROM orders WHERE customer_id IN (1,2,...,50)
// reducing 100 queries to 2 queries.

// ——— Fix 3: DTO Projection – only fetch needed data —————

interface CustomerOrderSummary {
    Long getCustomerId();
    String getCustomerName();
    Long getOrderCount();
}

```

```

    Double getTotalRevenue();
}

// In repository:
// @Query("""
//     SELECT c.id AS customerId, c.name AS customerName,
//           COUNT(o.id) AS orderCount, SUM(o.totalAmount) AS totalRevenue
//     FROM Customer c LEFT JOIN c.orders o
//     GROUP BY c.id, c.name
// """)
// List<CustomerOrderSummary> findCustomerOrderSummaries();

```

// — Test to detect N+1 (using datasource-proxy or similar) —————

```

class N1DetectionExample {
    /**
     * In a real test with datasource-proxy or p6spy:
     *
     * @Test
     * @Transactional
     * void loadingCustomersShouldNotTriggerNPlusOneQueries() {
     *     // seed 10 customers with 3 orders each
     *     QueryCountHolder.resetCount();
     *     List<Customer> customers = customerRepo.findAllWithOrders();
     *     customers.forEach(c -> c.getOrders().size()); // access collections
     *     assertThat(QueryCountHolder.getCount()).isLessThanOrEqualTo(2);
     *     // 1 SELECT customers, 1 JOIN to orders – never 11+ queries
     * }
    */
}

```

Common Interview Question & Expert Answer

Q: "What is the N+1 problem? Can `@EntityGraph` solve it for a deep nested association like `Customer → Orders → OrderItems`?"

A: The N+1 problem is when loading N parent entities leads to N additional queries for each parent's lazy children, totaling N+1 queries instead of 1.

`@EntityGraph` for nested associations: Yes, `@EntityGraph` can handle multiple nesting levels using dot notation:

```
java
```

```
@EntityGraph(attributePaths = {"orders", "orders.orderItems"})
Optional<Customer> findByIdWithOrdersAndItems(Long id);
```

This generates a JOIN that fetches `Customer → Orders → OrderItems` in a single query. **However**, there's an important caveat: when using `JOIN FETCH` on multiple `@OneToMany` collections simultaneously (e.g., both `orders` and `addresses` at the top level), Hibernate issues a warning about **"HHH90003004: firstResult/maxResults specified with collection fetch; applying in memory"** — this means Hibernate loads ALL rows into memory first and then paginates in Java, which defeats pagination entirely.

The correct solution for paginated queries with multiple collection fetches:

1. Fetch IDs first with pagination: `SELECT c.id FROM Customer c ORDER BY c.id LIMIT 20 OFFSET 0`.
2. Then fetch the full graph: `SELECT c FROM Customer c LEFT JOIN FETCH c.orders WHERE c.id IN :ids`.

This "two-query" pattern (Hibernate's **"count + offset"** pattern or Blaze-Persistence's approach) is the production-correct way to combine pagination with collection fetching.

6.12 Spring Boot Auto-Configuration

Clear Definition & Internal Mechanics

Auto-configuration is Spring Boot's mechanism for automatically configuring Spring beans based on what's on the classpath, what beans are already defined, and what properties are set.

Mechanism:

1. `@SpringBootApplication` includes `@EnableAutoConfiguration`.
2. Spring Boot loads all classes listed in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (Spring Boot 2.7+; previously `spring.factories`).
3. Each auto-configuration class is annotated with `@AutoConfiguration` and conditional annotations.
4. Conditions are evaluated:
 - `@ConditionalOnClass(DataSource.class)`: Only configure if `DataSource` is on the classpath.
 - `@ConditionalOnMissingBean(DataSource.class)`: Only configure if no `DataSource` bean is already defined.

- `@ConditionalOnProperty("spring.datasource.url")`: Only configure if the property is set.
- `@ConditionalOnWebApplication`: Only in web contexts.

5. Qualifying auto-configurations register their beans.

6. `application.properties` / `application.yml` externalize configuration properties into `@ConfigurationProperties`-bound POJOs.

Override Strategy: Auto-configured beans are **always lower priority** than user-defined beans. Defining your own `DataSource @Bean` in a `@Configuration` class automatically disables the auto-configured one (via `@ConditionalOnMissingBean`). This is the "**convention over configuration**" principle.

Debugging Auto-Configuration:

```
--debug flag: java -jar app.jar --debug
# Prints "CONDITIONS EVALUATION REPORT" showing which auto-configurations
# were applied, which were not, and why.
```

Real-Life Analogy

Auto-configuration is like a **smart hotel concierge who pre-configures your room** based on your guest profile. Before you arrive (at application startup), the concierge checks: "This guest usually takes coffee (has Jackson on classpath) → pre-set the coffee maker. This guest has their own alarm clock (defines custom `DateTimeFormatter` bean) → don't set the hotel's alarm clock." You get a fully set-up room without asking for each item individually. But you can always override: bring your own pillow, and the concierge won't replace it.

Interview Context & "Why it Matters"

Auto-configuration is what makes Spring Boot dramatically simpler than plain Spring. Understanding *how* it works allows you to: debug missing beans, exclude unwanted auto-configurations (`@SpringBootApplication(exclude=DataSourceAutoConfiguration.class)`), and write custom auto-configuration for internal libraries.

Production-Ready Code Snippet

```
java
```

```
import org.springframework.boot.autoconfigure.AutoConfiguration;
import org.springframework.boot.autoconfigure.condition.*;
import org.springframework.boot.context.properties.*;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;
```

// — Custom Properties —

```
@ConfigurationProperties(prefix = "app.notification")
```

```
@Validated
```

```
public class NotificationProperties {
```

```
    @NotBlank
```

```
    private String provider = "email";    // default
```

```
    @Positive
```

```
    private int maxRetries = 3;
```

```
    private boolean enabled = true;
```

```
    public String getProvider() { return provider; }
```

```
    public void setProvider(String provider) { this.provider = provider; }
```

```
    public int getMaxRetries() { return maxRetries; }
```

```
    public void setMaxRetries(int maxRetries) { this.maxRetries = maxRetries; }
```

```
    public boolean isEnabled() { return enabled; }
```

```
    public void setEnabled(boolean enabled) { this.enabled = enabled; }
```

```
}
```

// — Notification Service Interface —

```
interface NotificationSender {
```

```
    void send(String to, String message);
```

```
}
```

```
class EmailNotificationSender implements NotificationSender {
```

```
    private final NotificationProperties props;
```

```
    EmailNotificationSender(NotificationProperties props) { this.props = props; }
```

```
    @Override
```

```
    public void send(String to, String message) {
```

```
        System.out.printf("[Email] To=%s Msg=%s Retries=%d\n", to, message, props.ge
```

```
    }
```



```

}

class SmsNotificationSender implements NotificationSender {
    @Override
    public void send(String to, String message) {
        System.out.printf("[SMS] To=%s Msg=%s\n", to, message);
    }
}

// — Custom Auto-Configuration (library authors write this) —————

@Configuration
@EnableConfigurationProperties(NotificationProperties.class)
@ConditionalOnProperty(prefix = "app.notification", name = "enabled",
    havingValue = "true", matchIfMissing = true)
public class NotificationAutoConfiguration {

    // Only create EmailNotificationSender if no NotificationSender bean exists
    @Bean
    @ConditionalOnMissingBean(NotificationSender.class)
    @ConditionalOnProperty(prefix = "app.notification", name = "provider", havingValue = "email")
    public NotificationSender emailNotificationSender(NotificationProperties props) {
        System.out.println("[AutoConfig] Creating EmailNotificationSender (auto-config)");
        return new EmailNotificationSender(props);
    }

    @Bean
    @ConditionalOnMissingBean(NotificationSender.class)
    @ConditionalOnProperty(prefix = "app.notification", name = "provider", havingValue = "sms")
    public NotificationSender smsNotificationSender() {
        System.out.println("[AutoConfig] Creating SmsNotificationSender (auto-config)");
        return new SmsNotificationSender();
    }
}

// — User Override: Providing a custom bean disables auto-configuration —————
// @Configuration
// public class CustomConfig {
//     @Bean
//     public NotificationSender myCustomSender() {
//         return (to, msg) -> System.out.println("[CUSTOM] " + to + ": " + msg);
//     }
// }
// // ↑ @ConditionalOnMissingBean in auto-config detects this bean —

```

```
//      //  neither emailNotificationSender nor smsNotificationSender is created.  
// }
```

Common Interview Question & Expert Answer

Q: "How would you disable a specific Spring Boot auto-configuration, and why might you need to?"

A: Two methods:

1. Annotation exclusion (preferred for permanent, intentional disabling):

```
java  
  
@SpringBootApplication(exclude = {  
    DataSourceAutoConfiguration.class,  
    DataSourceTransactionManagerAutoConfiguration.class,  
    HibernateJpaAutoConfiguration.class  
})
```

2. Property exclusion (useful in `application.properties` for environment-specific control):

```
properties  
  
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.DataSourceA
```

Why you'd need to:

- Your app uses a **non-relational database** (MongoDB only), so `DataSourceAutoConfiguration` fails to find a JDBC URL and throws a startup error.
- You use a **custom DataSource configuration** (connection pool with specialized SSL) and don't want the default HikariCP auto-config interfering.
- **Testing:** In integration tests, you want to exclude `SecurityAutoConfiguration` to test business logic without authentication overhead.
- **Multi-module projects:** A library module includes Spring auto-config for convenience, but a consumer module wants to opt out and configure manually.

Always pair exclusion with the `--debug` flag output to confirm the auto-config is excluded and no secondary auto-configs depend on it (the CONDITIONS EVALUATION REPORT shows excluded configs clearly).